

Reviewer Pack — Minimal Rank of Free Probability Representations over XOR-AN...

Entry 6b1c0e45c9d3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 20:32:55 UTC

Minimal Rank of Free Probability Representations over XOR-AND Tree Width

Entry ID: 6b1c0e45c9d3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 20:32:55 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

{‘text’: ‘For every read-twice Boolean function f with XOR-AND tree width w , there exists a positive integer $r(f)$ such that the minimal rank of the free probability representation of f is at most $\Theta(r(f)) \log(w)$.’}

Rationale (proposer’s reasoning):

{‘text’: ‘Free probability theory provides a framework for studying random matrix models and their applications to various fields. This conjecture suggests that the minimal rank of free probability representations could serve as an invariant for XOR-AND tree width, potentially leading to new insights into the structure of Boolean functions and their complexity.’}

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6ca07339af5faef6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of randomly generated read-twice Boolean functions with XOR-AND tree width up to $n=40$ have their minimal rank of free probability representation correlated with their tree width by a Pearson correlation coefficient ≥ 0.8 , and the mean minimal rank does not exceed $\Theta(n \log(2n))$ for all functions.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - free probability theory AND (XOR-AND tree width OR tree-width XOR-AND) AND minimal rank - minimal rank in free probability representations AND (Boolean function OR read-twice Boolean functions) AND XOR-AND tree width - (free probability theory + free-probability + representation) AND (XOR-AND tree-width + complexity theory + Boolean function)

Top relevant hits considered: - [<http://arxiv.org/abs/2308.14677v4>] Twin-width of graphs with tree-structured decompositions - [<http://arxiv.org/abs/1204.6522v3>] Formal Groups, Witt vectors and Free Probability - [<http://arxiv.org/abs/2205.08916v3>] Monoidal Width: Capturing Rank Width - [<http://arxiv.org/abs/1305.0651v1>] Associative and commutative tree representations for Boolean functions - [<http://arxiv.org/abs/1609.03734v1>] Hacking of the AES with Boolean Functions - [<http://arxiv.org/abs/cs/0107010v1>] Algorithms for Boolean Function Query Properties - [<http://arxiv.org/abs/math/9810047v2>] The linearization of the central limit operator in free probability theory - [<http://arxiv.org/abs/hep-th/9707234v2>] Variational Approach to Quantum Field Theory: Gaussian Approximation and the Perturbative Expansion around It

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_read_twice_boolean_function(n):
    if n == 1:
        return [random.choice([0, 1])]
    left = generate_read_twice_boolean_function(n // 2)
    right = generate_read_twice_boolean_function(n - len(left))
    return [left[i] ^ right[i % len(right)] for i in range(n)]

def compute_free_probability_representation(f):
    n = len(f)
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if f[i] == 1 and f[j] == 1:
                A[i][j] += 1
            elif f[i] == 0 and f[j] == 0:
                B[i][j] += 1
    return A, B
```

```

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    rank = 0
    for row in A:
        if any(row):
            rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    ranks = []
    tree_widths = []

    for n in n_values:
        f = generate_read_twice_boolean_function(n)
        A, B = compute_free_probability_representation(f)
        rank_A = gaussian_elimination(A)
        rank_B = gaussian_elimination(B)
        min_rank = min(rank_A, rank_B)
        ranks.append(min_rank)
        tree_widths.append(n)

    correlation_coefficient = sum((ranks[i] - mean(ranks)) * (tree_widths[i] - mean(tree_widths)))

    conjecture_holds = correlation_coefficient >= 0.8 and mean(ranks) <= n_values[-1] * math.log(2)
    counterexample = "" if conjecture_holds else "correlation_coefficient=<{}> mean_rank=<{}>".format(correlation_coefficient, mean(ranks))

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

def mean(lst):
    return sum(lst) / len(lst)

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:

```


11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11009
2	propose	ollama_remote	glm4:latest	0	0	9396
3	preregistration	ollama_remote	glm4:latest	0	0	5990
4	novelty	ollama_remote	glm4:latest	0	0	4990
5	novelty	ollama_remote	glm4:latest	0	0	5895
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11838
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11562
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8499
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11941
10	judge	ollama_remote	glm4:latest	0	0	20459

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 101579 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/6b1c0e45c9d3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6b1c0e45c9d3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6b1c0e45c9d3.tar.gz` (if generated)